



COMP4421 复习总整理

第一章

这份讲义的核心脉络非常清晰，主要围绕“图像是如何从物理世界的连续光信号，转化为计算机能够处理的离散数字矩阵的”这一逻辑展开。为了配合你有限的复习时间，我将为你滤除边缘信息，直接拆解讲义中最核心的四个板块，并详细剖析其中的数学公式。

以下是为你整理的关键知识点：

一、 图像的物理获取与传感器阵列

数字图像的起点是物理世界的能量（光）。这需要通过传感器（如CCD）来完成能量的捕获与转换。

- **光电位点 (Photosite):** 每一个光电位点就是一个硅成像元件，它吸收光能并输出与光照强度成正比的电压。
- **传感器维度:** 传感器可以是单点、线性阵列 (Line scan sensor, 用于扫描仪, 生成1D信号)，或者是面阵列 (Area scan sensor, 用于相机, 由多行光电位点组成, 生成2D信号)。正如高精度的触觉测试台会利用传感器矩阵将物理接触转化为供支持向量机 (SVM) 分类的数据矩阵一样，面阵列光学传感器也会将二维的光学信号转化为数字图像矩阵。

二、 核心公式：连续图像函数

讲义中最重要且必须理解的公式是二维连续图像函数： $f(x, y) = i(x, y)r(x, y)$ 。这是一个用来描述图像上任意一点 (x, y) 的光照强度（亮度/颜色）的数学模型。

- x 与 y : 代表空间坐标系中的横纵位置。
- $f(x, y)$ (**Intensity**): 代表该坐标点上最终呈现的图像强度（亮度）。
- $i(x, y)$ (**Illumination - 光照分量**): 代表入射到该物体表面的光源能量大小。它的取值范围是 $0 \leq i(x, y) < \infty$ （即从完全无光到无限亮）。例如，晴天的光照分量极高，而夜晚办公室的光照分量则很低。
- $r(x, y)$ (**Reflectance - 反射分量**): 代表物体表面反射光线的能力，这是物体材料本身的固有物理属性。它的取值范围是 $0 \leq r(x, y) \leq 1$ 。其中 0 代表完全吸

收（纯黑），1 代表完全反射。在开发材料分类系统时，将外部的 $i(x, y)$ 噪声与材料内在的 $r(x, y)$ 特征解耦，是提取有效信息的关键。

公式逻辑总结： 你在一张照片上看到的某个像素有多亮 $f(x, y)$ ，取决于当时的环境有多亮 $i(x, y)$ ，以及那个物体本身的材质能反射多少光 $r(x, y)$ 。

三、从连续到离散：采样与量化

物理世界的光信号是连续的，但计算机只能处理离散的数字。将连续图像 $f(x, y)$ 转化为数字图像，必须经过两个关键步骤：

- **空间坐标的离散化 —— 采样 (Sampling)：**
 - 采样决定了图像的**空间分辨率 (Image resolution)**。
 - 它将连续的画面切分成网格，就像线性代数中的矩阵一样，拥有 n 行和 m 列。矩阵中的每一个独立元素被称为像素 (pixels)。
 - **后果：** 降低空间分辨率（减少采样的网格数量），图像就会出现马赛克，变得越来越模糊 (Pixelated)。
- **亮度的离散化 —— 量化 (Quantization)：**
 - 量化决定了图像的**灰度级数 (Grey level, L)**。
 - 原本连续的亮度值被强制分配到有限的几个挡位中。例如，如果用4个比特 (bits) 来表示，就会有16个灰度级；用8个比特，就会有256个灰度级。
 - **后果：** 保持空间分辨率不变，但减少灰度级数，图像中原本平滑过渡的色彩会出现明显的断层 (False contouring)。

四、存储空间计算

当图像被完全离散化为一个矩阵后，讲义提供了一个计算该图像占用内存大小的简单公式：

内存大小 = $M \times N \times$ 位深

- M = 图像的列数（宽度）
- N = 图像的行数（高度）
- 位深 (bits required to represent the grey levels) = 表示每一个像素灰度级所需的比特数。

第二章 Part1

收到。这份讲义进入了图像处理的实际操作环节，核心主题是**空间域内的图像增强（特别是灰度变换）**。简单来说，就是如何通过数学映射关系，直接修改图像中每一个像素的明暗程度，从而改善视觉效果。

为了让你在有限的时间内快速掌握，我为你提炼了以下四个最核心的考试与理解重点：

一、空间域变换的基础模型

- **一般公式：** $g(x, y) = T[f(x, y)]$
 - $f(x, y)$ 是输入图像， $g(x, y)$ 是输出图像， T 是操作算子。
- **点处理（灰度变换）简化公式：** $s = T(r)$
 - 当操作仅针对单个像素（ 1×1 邻域）时，我们不需要管坐标了。输入像素的灰度值记为 r ，输出的灰度值记为 s 。
 - **核心逻辑：** 整个讲义后面所有的公式，本质上都是在设计不同的 T ，告诉计算机“遇到亮度为 r 的点，把它变成亮度为 s 的点”。

二、三大基础灰度变换公式（重点）

这是讲义的核心，需要理解每种公式的物理意义和适用场景：

- **1. 图像反转 (Image Negatives)**
 - **公式：** $s = (L - 1) - r$
 - L 是灰度级数（如8位图的 $L = 256$, $L - 1 = 255$ ）。
 - **作用：** 类似以前的胶片底片，黑变白、白变黑。
 - **应用场景：** 当图像中大面积是暗色，而你需要观察的微小细节是白色或灰色时（例如某些医学影像分析），反转后人眼更容易识别。
- **2. 对数变换 (Log Transformations)**
 - **公式：** $s = c \log(1 + r)$
 - **作用：** 扩展低灰度值（暗部），压缩高灰度值（亮部）。
 - **应用场景：** 能大幅度提亮极暗的图像，揭示暗部细节。常用于显示傅里叶频谱等动态范围极大的数据。
- **3. 幂律变换 / 伽马变换 (Power-Law / Gamma Transformations)**
 - **公式：** $s = cr^\gamma$

- **作用：** 极其常用，也就是显示器和图像处理软件中常见的“伽马校正”。效果取决于指数 γ 的大小：
 - $\gamma < 1$ ：效果类似对数变换，整体**提亮**图像，拉伸暗部细节（例如处理因背光而显得暗淡的骨骼 MRI 图像）。
 - $\gamma > 1$ ：整体**压暗**图像，拉伸亮部细节（例如处理过度曝光的航拍图，让发白的地面显现出纹理）。

三、分段线性变换 (Piecewise-Linear Transformations)

不用单一公式，而是根据灰度区间的不同，采用不同的线性映射。

- **对比度拉伸 (Contrast Stretching)：**
 - 将原本集中在某个窄范围内的灰度值，强制拉伸到完整的灰度范围 $[0, L - 1]$ 。
 - **极端情况（阈值化 Thresholding）：** 当映射关系变成一条垂直的阶跃线时，图像就变成了只有纯黑和纯白的二值图像。
- **灰度级切片 / 窗口化 (Intensity-level Slicing)：**
 - 专门“高亮”特定范围 $[A, B]$ 内的灰度值。
 - **应用场景：** 医学图像分割。例如在血管造影中，已知含有造影剂的血管灰度值在一个特定区间，通过切片操作就可以让血管高亮显示，同时把其他无关组织全部变成黑色屏蔽掉。

四、比特平面切片与数字水印 (Bit-Plane Slicing)

这是将一个像素的值（例如8个比特）在物理存储层面上拆解开来看。

- **原理解析：**
 - 第8位 (**MSB**，最高有效位)：决定了图像绝大部分的视觉结构和对比度。
 - 第1位 (**LSB**，最低有效位)：包含了非常微小的细节变化，看起来像无规则的噪点。
- **工程应用（数字水印）：** 正因为修改底层的比特位（如第1到第3位）对人眼视觉影响极小，因此通常被用来隐藏秘密信息或嵌入数字水印，既达到了标记目的，又不会破坏原图的观感。

第二章 Part2

理解你的时间很紧张。这份讲义的核心是**空间域的图像增强：直方图处理 (Histogram Processing)**。这一章在图像处理中非常关键，主要是通过调整像素的亮度分布来提升图像的视觉效果。

我为你提炼了这份讲义中最核心的四大知识块，并把复杂的公式翻译成了通俗易懂的逻辑。

1. 什么是图像直方图 (Image Histogram)?

直方图本质上就是一个**统计图表**，它记录了整张图像中各个灰度级（从纯黑到纯白）分别有多少个像素。

- **核心公式解释：** 讲义中的 $p(r_k) = n_k/N$ 是最基础的公式。这里的 N 是图像的总像素数， n_k 是拥有特定灰度级 r_k 的像素数量。这个公式就是在计算“**概率**”，即某个灰度（比如中度灰）在整张照片中占的百分比。
- **直方图的作用：** 通过看这个图的形状，你就能一眼看出图像是太暗、太亮、对比度太低，还是对比度很高。
- **关键考点（盲区）：** 直方图**不包含任何空间信息**。也就是说，它只知道“有多少个黑点”，但不知道“这些黑点分布在图像的哪个位置”。因此，两张完全不同的图像，可能拥有完全相同的直方图。

2. 全局直方图均衡化 (Global Histogram Equalization) - 核心重点

这是讲义中最重要的算法。有时候图像对比度很低（灰蒙蒙的），直方图均衡化就是为了解决这个问题。

- **它的目的：** 把原本挤在一起的像素灰度，强制拉伸并均匀分布到整个灰度范围（从最黑到最白）。理想情况下，它希望每个灰度级上的像素数量都差不多（即达到均匀分布）。
- **公式大白话翻译：** 讲义中离散情况的公式为： $s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j)$ 。
不要被求和符号 $\sum$ 吓到，这其实是一个**累加 (Cumulative)** 的过程。假设图像最高灰度级是 7 ($L - 1 = 7$)，你要计算旧灰度级“3”转换后的新灰度级，你只需要把旧图中灰度级为 0、1、2、3 的概率全部加起来，然后乘以 7 即可。它就是一种根据前面积累的像素比例，来决定当前像素该被分配到多亮的位置的算法。
- **两个必须满足的数学条件：**
 1. **单调非递减：** 公式必须是持续上升或平坦的 ($T(r)$ 是单调非递减的)。这是为了保证原本暗的像素处理后依然相对暗，不能黑白颠倒。

2. **值域限制**：转换后的结果必须在合法的灰度范围内（比如 0 到 255）。

3. 局部直方图处理 (Local Histogram Equalization)

全局均衡化虽然好，但它是基于整张图算出来的，容易忽略掉图像中某些小区域的细节。

- **解决方法**：不再拿整张图计算，而是拿一个小方框（比如 3×3 的矩阵）在图像上逐个像素滑动。每次只针对框内的像素计算直方图并进行均衡化，然后把算出的结果赋值给框中心的那个像素。
- **效果**：这种方法能极其夸张地把局部区域隐藏的纹理和细节凸显出来。

4. 直方图规定化/匹配 (Histogram Specification / Matching)

均衡化的目标永远是把直方图变“平坦”，但有时我们不想让它平，而是想让它的直方图变成我们指定的一种特定形状（比如为了让两张照片的色调保持一致）。

- **逻辑（三步走）**：
 1. 先把原图像进行标准的“直方图均衡化”，得到一个中间值。
 2. 把你指定的“目标直方图”也假装做一次均衡化，计算出它的累加概率函数 $G(z)$ 。
 3. **逆向查找**：让原图均衡化后的像素值，去对应目标直方图均衡化函数 $G(z)$ 中最接近的值，从而找到最终的输出灰度级。

这四个部分构成了整篇讲义的逻辑主线。关于全局直方图均衡化的具体计算步骤，你需要我用讲义里那个 64×64 像素的例子带你手算一遍吗？

第二章 Part3

我很理解在有限时间内消化包含大量公式的专业讲义是很有挑战性的。我会为你剥离掉复杂的数学推导，直接提炼出这份讲义最核心的逻辑和算法的物理意义。无论是处理摄像头的视觉画面，还是处理高精度阵列触觉传感器的原始信号，这些空间域的滤波技术都是解耦噪声、提取有效特征的基础。

以下是这份讲义中你需要掌握的关键知识点：

1. 基础图像操作 (Basic Image Operations)

这些是最底层的像素级数学运算，用于实现最基础的增强目的。

- **减法操作 (Subtraction)**: 常用于检测两幅图像之间的差异，或者用于发现场景中的动态变化。
- **图像平均 (Image Averaging)**: 如果在相同条件下连续获取多张包含随机噪声的图像，将它们全部相加并取平均值，可以极其有效地抵消随机噪声，还原出更清晰的真实信号。

2. 空间滤波的核心机制 (Spatial Filtering)

- 空间滤波的核心是使用一个被称为“掩膜” (Mask) 或“核” (Kernel) 的小矩阵在整个图像的像素网格上逐个滑动。
- 在滑动到的每一个位置，掩膜内的权重系数会与下方对应的图像像素值一一相乘并求和，计算出的结果将成为该中心位置的新像素值。

3. 平滑滤波器 (Smoothing Filters)(线性滤波)

平滑滤波主要用于[去除随机噪声或进行模糊预处理](#)，它们被称为“[低通滤波器](#)”，因为它们允许低频信号（平缓变化的区域）通过，而过滤掉高频信号（尖锐的边缘和噪声点）。

- **均值滤波 (Average/Box Filter)**: 最基础的平滑方式，掩膜内的系数全部为正数且相等，相当于直接计算局部邻域像素的平均值。
- **高斯滤波 (Gaussian Smoothing)**: 使用二维高斯分布作为权重，其核心思想是[距离中心越近的像素，对最终结果的影响权重越大](#)。
- 高斯滤波的数学表达式为 $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ 。
- 在实际应用中，公式中的 σ 控制着模糊的剧烈程度，该参数越大，图像平滑的效果越明显，但也意味着会丢失更多的边缘细节。

4. 非线性滤波与中值滤波 (Non-linear & Median Filter)

- 与基于加权求和的线性滤波不同，非线性滤波器主要通过对局部邻域内的像素值进行排序来实现增强。
- **中值滤波 (Median Filter)**: 它将掩膜覆盖区域内的所有像素值按大小排序，然后直接提取最中间的那个数值作为输出。
- 中值滤波在[去除“椒盐噪声” \(极亮或极暗的脉冲噪点\)](#)方面表现极其优异，且与低通滤波器相比，它能够在去噪的同时极好地保留图像的锐利边缘。
- 会收敛

5. 锐化滤波器 (Sharpening Filters)

锐化的目的是突出图像中的精细细节或补偿被模糊的边缘，这被称为“**高通滤波**”，因为它滤除了平缓区域，只保留了变化剧烈的部分。

- **一阶导数 (Sobel 算子)**: 通过计算相邻像素之间的强度差值（梯度）来检测边缘的位置，Sobel 算子在计算差值时会对靠近中心的邻居赋予更大的权重。
- **二阶导数 (Laplacian 算子)**: 用于突出图像中强烈的强度过渡区域，其拉普拉斯算子公式表达为 $\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$ 。
- 拉普拉斯算子的掩膜同时包含正系数和负系数，且系数总和为零，因此当它应用在纯色或恒定强度区域时，输出结果为零，只在边缘处产生强烈响应。
- **反锐化掩蔽 (Unsharp Masking)**: 一种经典的锐化流程。首先生成原图的模糊版本，接着用原图减去模糊版本以提取出高频细节，最后将这些细节叠加回原图中，从而提升整体锐度。

6. 双边滤波 (Bilateral Filtering)

- 双边滤波是一种非常强大的现代技术，能够在平滑图像消除噪声的同时，极其严格地保留物体边缘。
- 它的巧妙之处在于结合了两种权重：**空间权重**（距离越近的像素权重越大）和**范围/强度权重**（像素亮度值越接近的权重越大）。
- 这种机制确保了只有在空间位置相近且自身数值也相似的像素才会参与平均计算，有效防止了跨越物体边缘进行平滑而导致的轮廓模糊现象。

针对你目前手头需要处理的信号或图像任务，你更倾向于重点解决异常噪声点的问题，还是希望尽可能清晰地提取和强化目标边缘轮廓？

第三章

理解你在有限时间内掌握这份讲义的压力。频域图像处理的数学确实容易让人望而生畏，但剥开复杂的微积分，其核心逻辑其实非常直观。

为了让你能够快速消化，我将跳过繁琐的数学推导，为你梳理这份讲义中最核心的知识骨架。

1. 核心思想：什么是频域？

在处理图像时，我们通常看的是**空间域 (Spatial Domain)**，也就是由一个个像素点组成的画面。而**频域 (Frequency Domain)** 提供了一个全新的视角：它不关心像素在

哪里，只关心图像的灰度值**变化有多快**。

- **低频分量 (Low Frequencies):** 对应图像中灰度变化缓慢的区域，比如大面积的平滑背景或物体的整体轮廓。
- **高频分量 (High Frequencies):** 对应图像中灰度变化剧烈的区域，比如物体的边缘、细节，以及随机产生的噪点。

2. 频域滤波的四大黄金步骤

整个讲义中所有的滤波操作，都在重复这四个步骤：

1. **预处理与傅里叶变换:** 将输入图像 $f(x, y)$ 通过离散傅里叶变换 (DFT) 转换到频域，得到 $F(u, v)$ 。
2. **构建滤波器:** 设计一个滤波器函数 $H(u, v)$ 。
3. **频域相乘:** 将滤波器 $H(u, v)$ 与图像的频谱 $F(u, v)$ 进行逐元素相乘得到 $G(u, v)$ 。公式非常简单: $G(u, v) = F(u, v)H(u, v)$ 。
4. **反傅里叶变换:** 将处理后的频谱 $G(u, v)$ 通过反傅里叶变换转回空间域，得到最终的增强图像 $g(x, y)$ 。

3. “天书”公式的直白翻译

讲义中出现了大量公式，你只需要重点理解以下几个：

- **二维离散傅里叶变换 (2D DFT):**

$$F(u, v) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

一句话解释: 这是一个分解过程。它将空间域的像素 $f(x, y)$ 与不同的正弦/余弦波（由复指数 $e^{-j\cdots}$ 表示）相乘并求和。算出的 $F(u, v)$ 就是在特定频率 (u, v) 下，该波形在图像中所占的权重。

- **巴特沃斯低通滤波器 (BLPF):**

$$H(u, v) = \frac{1}{1 + [D(u, v)/D_0]^{2n}}$$

一句话解释: $D(u, v)$ 是当前点到中心的距离， D_0 是你设定的“截止半径”。当距离 D 远大于 D_0 （高频区）时，分母变得很大， H 趋近于0（阻挡信号）；当 D 远小于

D_0 （低频区）时， H 趋近于1（放行信号）。阶数 n 决定了阻挡和放行之间的过渡有多陡峭。

4. 两大核心任务：平滑与锐化

A. 平滑 (Smoothing) = 低通滤波 (Lowpass Filtering)

低通滤波器的作用是“放行低频，阻挡高频”，这就像在处理物理传感器数据时，为了获得平稳的操控信号，需要滤除掉高频的传感噪声一样。它能让图像变得平滑、模糊，并消除噪点。

讲义介绍了三种主要的低通滤波器：

- **理想低通滤波器 (ILPF)**：像一堵垂直的墙，截断极其生硬。**致命缺点**是会在图像中产生明显的“振铃效应” (Ringing artifact)，也就是图像边缘会出现水波纹般的伪影。
- **巴特沃斯低通滤波器 (BLPF)**：有一个平滑的过渡区，没有尖锐的不连续性。只要阶数 n 不太大，几乎看不出振铃效应。
- **高斯低通滤波器 (GLPF)**：极其平滑，**完全没有振铃效应**，是最常用的低通滤波器之一。

B. 锐化 (Sharpening) = 高通滤波 (Highpass Filtering)

高通滤波器的作用是“阻挡低频，放行高频”，用于突出边缘和细节。

- **与低通的关系**：它们是对立统一的。高通滤波器通常可以直接用 1 减去对应的低通滤波器得到： $H_{hp}(u, v) = 1 - H_{lp}(u, v)$ 。
- **高频强调滤波 (High-Frequency Emphasis)**：纯粹的高通滤波会把背景（低频）完全抹黑。为了保留背景同时锐化细节，我们可以在滤波器上加一个常数 a （保留部分低频），并乘上一个系数 b （放大高频），公式形如 $H(u, v) = a + b \cdot H_{hp}(u, v)$ 。

5. 高阶应用：同态滤波 (Homomorphic Filtering)

这是讲义最后提到的一个经典应用场景。一幅图像 $f(x, y)$ 可以看作是**光照 (Illumination)** 和**反射率 (Reflectance)** 的乘积： $f(x, y) = i(x, y)r(x, y)$ 。

- **问题**：傅里叶变换很难直接处理乘法。
- **破局点**：利用自然对数 (log) 将乘法变成加法： $\ln f = \ln i + \ln r$ 。
- **处理逻辑**：光照通常是缓慢变化的（低频），而物体的反射率（细节/纹理）变化很快（高频）。因此，在对数域进行傅里叶变换后，我们可以设计一个滤波器，

压低低频分量（均匀化光照），并提升高频分量（增强细节对比度）。最后再通过指数运算 (exp) 转换回来。

以上就是整个讲义的主干逻辑，理清这些之后再看具体的图片例子就会豁然开朗。关于这几种滤波器（理想、巴特沃斯、高斯），你是否需要我详细对比一下它们在实际图像处理中产生的视觉差异？

第四章 Part1

时间紧迫，我们直接切入正题。在处理传感器视觉数据或进行图像分析时，我们经常需要从充满干扰的真实环境中提取清晰信号。这份讲义的核心逻辑其实非常清晰，主要分为两个大方向：**第一，图像是如何被噪声污染或退化（模糊）的；第二，我们如何利用数学工具把它们还原（滤波与复原）。**

公式看起来复杂，但它们大多只是在描述“如何取平均”或“如何赋予不同像素权重”。我会为你过滤掉推导过程，直接提取最核心的公式物理意义和应用场景。

1. 核心模型：图像退化与复原

要理解整份讲义，首先要看懂这个贯穿始终的公式。图像在获取或传输时，会受到系统本身的模糊（比如镜头失焦、运动模糊）以及随机噪声的干扰。

在空间域（图像本身的像素点），这个过程被建模为：

$$g(x, y) = h(x, y) * f(x, y) + \eta(x, y)$$

- $f(x, y)$ ：我们想要得到的**完美原图**。
- $h(x, y)$ ：退化函数（比如模糊效果），* 代表卷积（你可以理解为一种涂抹操作）。
- $\eta(x, y)$ ：随机加进去的**噪声**。
- $g(x, y)$ ：我们最终看到的**损坏图像**。

我们的终极目标：已知 $g(x, y)$ ，尝试估算出一个最接近原图的 $\hat{f}(x, y)$ 。

2. 图像噪声模型 (Image Noise)

噪声是在图像获取（如光照不足、传感器温度）或传输过程中产生的随机干扰。你不需要记住所有概率密度函数（PDF），只需重点理解最常见的两种：

- **高斯噪声 (Gaussian Noise):** 最常见的自然噪声，呈现正态分布。通常由电子电路噪声引起。

- **椒盐噪声 (Impulse/Salt-and-Pepper Noise):** 图像上随机出现的纯黑点（胡椒, Pepper）和纯白点（盐, Salt）。

3. 空间域滤波 (Spatial Filtering)

当图像只有噪声，没有系统性模糊时，我们通常在像素周围画一个框（窗口 S_{xy} ），通过计算框内像素的某些统计值来替换中心像素，以此消除噪声。

均值滤波器 (Mean Filters)

这是一种线性平滑，通过求平均来模糊噪声，但代价是图像边缘也会跟着变模糊。

- **算术均值 (Arithmetic Mean):** 简单粗暴求平均，对高斯噪声有效。
- **几何均值 (Geometric Mean):** 把像素值乘起来开根号。平滑效果类似，但在处理过程中丢失的图像细节比算术均值少。
- **逆谐波均值滤波器 (Contraharmonic Mean Filter):** 这是一个带参数 Q 的强大公式：

$$\hat{f}(x, y) = \frac{\sum_{(s,t) \in S_{xy}} g(s, t)^{Q+1}}{\sum_{(s,t) \in S_{xy}} g(s, t)^Q}$$

- **关键点：** 它是通过改变 Q 的正负号来对付不同噪声的。
- 当 $Q > 0$ 时，可以消除**胡椒噪声**（黑点）。
- 当 $Q < 0$ 时，可以消除**盐噪声**（白点）。

统计排序滤波器 (Order-Statistics Filters)

不进行数学计算，而是把窗口内的像素排个序，挑一个出来。

- **中值滤波 (Median Filter):** 挑出中位数替换中心像素。
 - **关键点：** 它是去除**椒盐噪声**的绝对王者，并且比均值滤波器更能保留图像的锐利边缘。
- **Min-Max filter**
- **Mid-point filter**
- **Alpha-trimmed mean filter**

自适应滤波器 (Adaptive Filters)

前面的滤波器对全图一视同仁，而自适应滤波器会根据局部的特点（比如局部方差，即这里是不是边缘）改变策略。

- **自适应中值滤波:** 如果发现算出来的中位数本身就是个噪声（极值），它会自动**扩大窗口尺寸**继续找，直到找到正常的像素值为止，这样可以极大地保护非噪声的细节。
 - Step1: 判断中值是不是噪声，是，增大窗口，不是，进入Step2
 - Step2: 判断当前点 (x,y) 是不是噪声，是，用中值替换，不是，保留原值

4. 频率域滤波与图像复原 (Frequency Domain Restoration)

当遇到**周期性噪声**（比如机电干扰产生的条纹），或者图像发生了**模糊**，我们就需要把图像转换到频率域（通过傅里叶变换）来处理。

- **陷波滤波器 (Notch Filters):** 周期性噪声在频域中表现为非常亮的独立光斑。陷波滤波器的作用就像是在这些光斑上“打孔”，直接把特定频率的干扰挖掉。
 - Butterworth notch reject filters
 - Gaussian notch reject filters
- **带阻滤波器(Bandreject filters)**
 - Butterworth Bandreject Filter
 - Gaussian Bandreject Filter
- **带通滤波器(Bandpass Filters)**

解决模糊问题：逆滤波 vs. 维纳滤波

如果图像不仅有噪声，还被 $H(u, v)$ 模糊了，我们该怎么复原？

- **逆滤波 (Inverse Filtering):** 最直观的想法是，既然你乘了 H ，那除以 H 不就行了吗？

$$\hat{F}(u, v) = F(u, v) + \frac{N(u, v)}{H(u, v)}$$

- **致命缺陷:** 这个公式解释了为什么逆滤波在现实中没用。当频率域的某些地方 $H(u, v)$ 非常小（接近 0）时，噪声 N 除以一个极小的数，会导致噪声被无限放大，彻底毁掉图像。

- **最小均方误差滤波 (Wiener Filtering / 维纳滤波):** 这是整本讲义中最经典的复原算法。

$$\hat{F}(u, v) = \left[\frac{H^*(u, v)S_f(u, v)}{S_f(u, v)|H(u, v)|^2 + S_\eta(u, v)} \right] G(u, v)$$

- **关键点:** 公式看起来吓人，但它的核心逻辑是：在分母中加入了一个噪声相关项 $S_\eta(u, v)$ 。即使 H 接近 0，分母也不会变成 0，从而完美解决了逆滤波中噪声被无限放大的问题。

5. 几何变换 (Geometric Transformations)

最后这部分脱离了噪声，主要解决图像形状的扭曲（比如相机镜头的桶形畸变校正）。

它分为两步：

1. **空间变换:** 把扭曲图像的坐标 (x', y') 映射回正确的坐标 (x, y) 。通常需要借助已知坐标的“控制点”(Tiepoints) 来求解变换矩阵。
2. **灰度插值:** 映射回来的坐标往往不是整数（比如映射到了 1.5 像素的位置），这就需要插值。
 - **最近邻插值 (Nearest neighbor):** 简单找离得最近的整数像素的颜色赋给它。
 - **双线性插值 (Bilinear interpolation):** 用周围 4 个已知像素点的值，通过距离加权计算出一个新颜色，效果更平滑。

以上就是这份讲义的骨架。掌握了这些核心公式的物理意义，应对考试或实际工程运用就足够了。

你觉得在“空间域滤波（去噪声）”和“频率域滤波（去模糊/去条纹）”这两块中，哪一部分你需要我再结合具体的工程场景（比如摄像头标定或传感器信号去噪）深入讲解一下？

第四章 Part2

理解你的复习压力。这份讲义的核心逻辑其实非常清晰，它讲述了图像处理中如何解决一个经典矛盾：**既要去除图像的噪声，又要保留图像的锐利边缘。**

为了让你快速消化，我将跳过繁琐的推导，直接按照讲义的逻辑主线，为你拆解最重要的知识点和公式背后的直观物理意义。

第一步：问题的引入——高斯平滑（线性滤波）的局限性

讲义首先介绍了一种最基础的去噪方法，也是我们需要改进的“反面教材”：高斯滤波。

- **基本概念**：高斯滤波是一种低通滤波器，它通过对像素及其周围像素的强度值进行加权平均来平滑图像。
- **权重分配**：这种加权平均的权重是由高斯分布决定的，中心像素的权重最高，距离越远权重越低。
 - **公式含义**：讲义中给出的二维高斯函数为 $G_{\sigma}(\vec{x}) = \frac{1}{2\pi\sigma^2} \cdot \exp(-\frac{\|\vec{x}\|^2}{2\sigma^2})$ 。在这个公式中，最关键的变量是方差（或标准差） σ 。 σ 的值越大，或者滤波的时间 t 越长，图像的平滑效果就越强烈。
- **致命缺点**：虽然高斯平滑能减少噪声，但它会“无差别攻击”，把图像中重要的结构特征（比如边缘）也一起模糊掉，使得边缘难以被识别。更严重的是，随着平滑程度的增加，边缘的实际位置还会发生偏移。

第二步：物理学视角的引入——扩散方程

为了解决高斯滤波模糊边缘的问题，讲义引入了物理学中的“扩散（Diffusion）”概念，这是理解后续所有公式的桥梁。

- **物理类比**：你可以把图像的像素强度（浓度 u ）想象成房间里的热量。热量总是从高温区向低温区扩散。
- **核心方程**：讲义引入了热传导方程 $\frac{\partial u}{\partial t} = \text{div}(D\nabla u)$ 。我们来拆解这个看似复杂的公式：
 - $\frac{\partial u}{\partial t}$ 代表图像像素强度随着时间（在算法中表现为迭代次数）的变化率。
 - ∇u 代表图像的梯度，也就是相邻像素之间亮度的差异程度。梯度越大，说明这里大概率是一条边缘。
 - D 代表扩散张量（Diffusivity），它就像是一个水龙头，控制着扩散的速率和方向。
- **线性扩散与高斯滤波的等价性**：当 D 为单位矩阵（即常数）时，这就叫线性扩散滤波，方程简化为 $\frac{\partial u}{\partial t} = \Delta u$ 。从数学上来说，**线性扩散的求解结果完全等价于第一步提到的高斯平滑**。因此，线性扩散同样会模糊重要的边缘。

第三步：终极解决方案——非线性扩散滤波

这就是整份讲义的核心精华。既然常数 D 会导致无差别模糊，我们自然想到：能不能让 D 变得“聪明”一点？

- **设计思路**：我们希望在图像某个区域内部积极进行平滑（去噪），但当遇到区域边界（也就是边缘）时，平滑操作能自动停止。
- **非线性方程**：为了实现这一点，我们把常数 D 替换成一个依赖于图像梯度大小的函数 $g(\|\nabla u\|)$ 。现在的公式变成了 $\frac{\partial u}{\partial t} = \text{div}(g(\|\nabla u\|)\nabla u)$ 。
- **如何理解 g 函数**？讲义给出了两种常用的扩散控制函数，例如 $g(\|\nabla u\|) = \exp(-(\frac{\|\nabla u\|}{K})^2)$ 或 $g(\|\nabla u\|) = \frac{1}{1+(\frac{\|\nabla u\|}{K})^2}$ 。
 - **在平坦区域（噪声区）**：像素亮度变化很小，梯度 $\|\nabla u\|$ 接近 0。此时经过公式计算， g 的值接近 1。这相当于水龙头完全打开，让图像正常进行扩散平滑。
 - **在边缘区域**：像素亮度剧烈变化，梯度 $\|\nabla u\|$ 非常大。此时经过公式计算， g 的值会迅速趋近于 0。这相当于水龙头被强制关上了，扩散被阻止，从而完美地保护了边缘不被模糊。

总结

你只需要记住这条逻辑链：

1. **线性滤波（高斯平滑）** 相当于一根不管不顾的水管，把整张图像的细节和噪声一起冲刷模糊。
2. **非线性滤波** 给这根水管加了一个“智能阀门”(g 函数)。当它检测到图像梯度很大（遇到边缘）时，阀门自动关闭停止平滑；当梯度很小（遇到平坦噪声区）时，阀门打开进行平滑去噪。
3. 最终的实际效果是：不仅大幅降低了噪声，还能保留甚至增强图像的锐利边缘和重要结构。这种技术在医学超声图像去噪 以及血管等管状结构的增强中非常实用。

第五章

这份讲义的核心是**形态学图像处理 (Morphological Image Processing)**。这是一种基于几何形状来处理图像的技术，它的底层数学语言是“集合论”。

为了让你能快速消化，我们可以把讲义中的复杂公式转化为直观的物理动作。可以将所有形态学操作理解为：**使用一个特定形状的“刷子”或“探测器”（称为结构元，Structuring Element, 简称 SE），在整张图像上滑动，并根据“刷子”和图像的重叠情况来决定如何修改像素。**

以下是讲义中最核心的知识点和公式的直观拆解：

1. 基础操作：膨胀与腐蚀 (二值图像)

这是所有高级操作的基石。在二值图像中，通常用 1（白色）代表前景物体，用 0（黑色）代表背景。

- **膨胀 (Dilation)**

- **公式:** $A \oplus B = \{z | (\hat{B})_z \cap A \neq \emptyset\}$
- **直白讲解:** 将结构元 B 反转（如果是对称的则不变），然后在图像上滑动。只要结构元和目标物体有**哪怕一个像素的接触（重叠）**，滑动终点的位置就被标记为 1。
- **视觉效果:** 物体会被“吹掉”一层向外扩张，整体变胖。
- **核心用途:** 用来桥接断裂的字符，或者填补物体内部的小裂缝

- **腐蚀 (Erosion)**

- **公式:** $A \ominus B = \{z | (B)_z \subseteq A\}$
- **直白讲解:** 拿着结构元在图像上滑动。只有当整个结构元**完全被包围**在目标物体内部时，滑动终点的位置才被保留为 1。
- **视觉效果:** 物体会被“剥掉”一层皮，整体变瘦。
- **核心用途:** 用来消除图像中无关的微小细节或噪点（比结构元小的物体都会被彻底抹除）

2. 进阶操作：开运算与闭运算 (二值图像)

这两个操作是膨胀和腐蚀的组合，能够实现更精细的平滑处理。

- **开运算 (Opening)**

- **公式:** $A \circ B = (A \ominus B) \oplus B$
- **逻辑:** 先腐蚀，后膨胀。
- **直白讲解:** 想象拿着一个和结构元形状一样的“球”，在目标物体的**内部**边缘贴着滚一圈。球滚不到的那些细小尖刺或狭窄的连接处，就会被“削掉”。
- **视觉效果:** 平滑物体轮廓，断开狭窄的细颈，去除细小的凸出物。

- **闭运算 (Closing)**

- **公式:** $A \bullet B = (A \oplus B) \ominus B$
- **逻辑:** 先膨胀，后腐蚀。

- **直白讲解**：想象拿着这个“球”，在目标物体的**外部**边缘贴着滚一圈。球滚不进的那些细小凹陷、缝隙或孔洞，就会被填平。
- **视觉效果**：平滑物体轮廓，弥合狭窄的裂缝，填补细小的孔洞。

3. 四个极其重要的应用算法

讲义利用上述基础操作，推导出了几个解决实际问题的算法公式，掌握这些公式的逻辑非常关键：

- **边界提取 (Boundary Extraction)**

- **公式**： $\beta(A) = A - (A \ominus B)$
- **逻辑**：原图减去被腐蚀后的图。因为腐蚀会把物体剥掉一层，所以“原图”减去“剥掉皮剩下的芯”，自然就得到了物体的“边界”。

- **区域填充与连通分量 (Region Filling & Connected Components)**

- **核心逻辑**：以目标区域内的一个已知点为起点，不断向外“膨胀”。
- **限制条件（交集）**：为了防止膨胀无限蔓延到不需要的地方，每次膨胀后必须跟一个已知的边界求交集来限制范围。
 - 如果是填充孔洞：用背景 (A^c) 限制边界，公式即 $X_k = (X_{k-1} \oplus B) \cap A^c$ 。
 - 如果是提取整个相连的物体：用原图 (A) 限制边界，公式即 $X_k = (X_{k-1} \oplus B) \cap A$ 。
- **停止条件**：当连续两次迭代的结果一模一样时 ($X_k = X_{k-1}$)，算法结束。

- **骨架(Skeletons)**

- 在图形内部找一个以 z 为中心的最大圆。要求：圆不能超出区域 A ，并且尽可能大。如果这个圆满足：再也不能变大了，且碰到边界 ≥ 2 个地方。那么这个点就属于骨架。

- **击中-击不中变换 (Hit-or-Miss Transform)**

- **公式**： $I \otimes B_{1,2} = (A \ominus B_1) \cap (A^c \ominus B_2)$
- **逻辑**：这是一种形状匹配工具。它必须同时使用两个结构元： B_1 用来探测物体内部（必须“击中”目标）， B_2 用来探测物体外部环境（必须“击不中”周围背景）。只有两者同时满足（求交集），才算找到了你想要的特定形状。

4. 灰度形态学 (Gray-Scale Morphology)

讲义的后半部分将操作从非黑即白的图像推广到了有灰度过渡的普通照片。在灰度图里，形态学操作变成了求“局部最大值”或“局部最小值”。

- **灰度膨胀**：本质是在结构元覆盖的邻域内寻找**局部最大值**。处理后，图像整体会变亮，黑暗的细小细节会被抹去。
- **灰度腐蚀**：本质是在结构元覆盖的邻域内寻找**局部最小值**。处理后，图像整体会变暗，明亮的细小细节会被抹去。
- **形态学梯度 (Morphological Gradient)**：公式是 $g = (f \oplus b) - (f \ominus b)$ 。用膨胀后的图（亮区扩大）减去腐蚀后的图（暗区扩大），得到的结果能够完美提取出图像中灰度发生剧烈变化的边缘。

第六章

理解你的时间紧迫。这份讲义的核心逻辑非常清晰：它主要讲述了**颜色是什么、计算机如何表示颜色，以及如何通过数学操作来修改或提取颜色**。

数学公式往往只是在用严谨的符号描述几何图形或距离，我会为你剔除枯燥的部分，直接将核心概念和公式翻译成易懂的语言。以下是这份讲义中最关键的知识点梳理：

1. 颜色的基础与三大颜色模型

这是后续所有图像处理的基础。讲义主要介绍了三种模型：

- **RGB 模型（面向硬件设备）**：光的三原色是红 (Red)、绿 (Green)、蓝 (Blue)。在计算机显示器中，所有的颜色都是由这三种光叠加而成的。在三维坐标系中，它表现为一个立方体，黑色在原点 $(0, 0, 0)$ ，白色在离原点最远的对角 $(1, 1, 1)$ 。
- **CMY 模型（面向打印颜料）**：青色 (Cyan)、品红色 (Magenta) 和黄色 (Yellow)。颜料的显色原理是吸收光，例如青色颜料会吸收红光。它和 RGB 是互补互逆的关系。
- **HSI 模型（面向人类感知）**：这是讲义中最重要、应用最广的模型。人类描述颜色不会说“这里有30%的红和20%的绿”，而是将颜色拆分为三个独立的属性：
 - **色调 (Hue, H)**：代表具体是什么颜色，比如红色、黄色。
 - **饱和度 (Saturation, S)**：代表颜色的纯度。粉色就是一种饱和度较低的颜色，因为它混入了白光。
 - **亮度/强度 (Intensity, I)**：代表颜色的明暗程度。

- **重要公式拆解：** 将 RGB 转换为 HSI 为什么重要？因为你可以单独调整图像的明暗（亮度），而不改变它原本的颜色和纯度。

- 亮度公式：

$$I = \frac{1}{3}(R + G + B)$$

这个公式非常直白，亮度就是红绿蓝三个通道数值的平均分。

- 饱和度公式：

$$S = 1 - \frac{3}{R + G + B}[\min(R, G, B)]$$

解释一下：公式中的 $\min(R, G, B)$ 找出了 RGB 中数值最小（最弱）的那一个分量，这个最弱的分量代表了混入其中的“白光”含量。白光成分越多，饱和度 S 就越低。

2. 伪彩色图像处理 (Pseudocolor Image Processing)

- **核心目的：** 很多图像原本是黑白灰度的，人类肉眼对灰度的分辨能力有限，对颜色的分辨能力却极强。伪彩色处理就是人为地给灰度图像“上色”。
- **强度切片技术 (Intensity Slicing)：** 想象把一张灰度图看作高低起伏的山脉（三维地形图）。我们在特定的高度“切一刀”（设定一个亮度阈值），把高于这个高度的像素涂成颜色 A，低于的涂成颜色 B。这常用于医学成像或卫星云图中标识降雨量。

3. 全彩色图像处理与颜色切片 (Color Slicing)

全彩色图像处理是指直接对相机捕捉到的彩色图像进行操作。这里有一个非常核心且公式看起来最复杂的知识点：**颜色切片**。

- **核心目的：** 在一张复杂的彩色图片中，我们只想把特定颜色范围的物体凸显出来（比如找出一堆水果中所有的红草莓），而把背景全部变成统一的灰色。
- **公式拆解（两种切片方法）：**
 - **方法一：立方体切片法**

$$s_i = \begin{cases} 0.5 & \text{if } |r_j - a_j| > \frac{W}{2} \text{ for any } 1 \leq j \leq n \\ r_i & \text{otherwise} \end{cases}$$

通俗解释：假设我们想要寻找“标准红色”，我们把标准红色定为中心点 a 。然后围绕这个点画一个边长为 W 的隐形立方体。我们检查图像里的每一个像素点 r 。如果这个像素点的 RGB 值跑出了这个立方体的范围（数学表达就是绝对值差 $> \frac{W}{2}$ ），我们就判定它不是目标颜色，把它强制变成中性灰色（数值设为 0.5）。如果是目标颜色，就保持原样 r_i 。

- **方法二：球体切片法**

$$s_i = \begin{cases} 0.5 & \text{if } \sum_{j=1}^n (r_j - a_j)^2 > R_0^2 \\ r_i & \text{otherwise} \end{cases}$$

通俗解释：这里的公式其实就是最基础的“计算两点间直线距离”（空间勾股定理）。我们同样设定标准红色为中心点 a 。如果某个像素点和中心点的距离的平方，大于我们设定的球体半径的平方 R_0^2 ，说明颜色偏差太大，涂成灰色；否则保留原样。

4. 图像分割与空间滤波

- **空间滤波（平滑与锐化）**：当你想要让一张彩色照片变清晰（锐化，使用拉普拉斯算子计算 ∇^2 ）或者去噪（平滑，使用像素平均值 $\bar{c}(x, y)$ ）时，最佳实践是在 HSI 模型的亮度 (I) 分量上进行操作，处理完再转回 RGB，这样可以保证颜色本身不发生畸变。
- **图像分割**：就是把感兴趣的物体从背景中提取出来。讲义展示了如何利用颜色的平均值和标准差划定范围，从而在 RGB 空间中成功分割出特定的红色区域。

这几大模块构成了讲义的基础骨架。针对你的考试或应用需求，你目前最希望我深入讲解 HSI 模型的具体转换应用，还是图像分割与滤波的操作逻辑？

第七章 Part1

1. 图像分割的核心目标与思路

- 图像分割的目的，是将图像细分成不同的构成区域，以提取有用的物体。
- 实现分割通常基于图像灰度值的两个基本特性：“不连续性”（如突然变化的边缘）和“相似性”（如颜色一致的平滑区域）。
- 本讲义的核心重点是基于“不连续性”的检测，也就是如何在图像中找到孤立的点、线和边缘。

2. 边缘检测的数学直觉（告别死记硬背）

寻找边缘，本质上就是寻找图像里“灰度变化最剧烈”的区域。讲义使用了微积分中的导数概念来实现这一点：

一阶导数（梯度）：用来判断“是否存在边缘”。梯度的幅值越大，说明灰度变化越快，该位置是边缘的概率就越高。

二阶导数：用来精确定位边缘的中心点，并判断边缘的明暗过渡方向。二阶导数穿过零点的位置（即零交叉点，Zero-crossing），就是较宽边缘的最中心。

核心痛点：导数计算对图像中的噪声极其敏感，一点点噪声就会导致导数失真。因此，在进行边缘检测之前，**必须先**对图像进行平滑（去噪）处理。

3. 常考的经典算子（空间滤波器）

讲义中提到了多种算子，你可以把它们理解为在图像上滑动的 3×3 矩阵模板，用于计算局部的像素差异：

Roberts, Prewitt, Sobel 算子：这些都是一阶导数（梯度）算子，用于检测水平、垂直或对角线方向的边缘。

LoG (Laplacian of Gaussian) 算子：这是一个二阶导数算子。它的精妙之处在于将两步合二为一：先用高斯函数（Gaussian）平滑图像消除噪声，然后再用拉普拉斯算子（Laplacian）寻找零交叉点，从而非常可靠地定位边缘。

4. 重点必考：Canny 边缘检测器

Canny 是目前最流行且性能最优的边缘检测算法之一。你需要清晰地记住它按顺序执行的三个主要步骤：

- **第一步：计算梯度。**如同基础方法一样，先求取图像平滑后的梯度幅值和方向。
- **第二步：非极大值抑制 (Non-maximum suppression)。**这一步是为了将粗糙、模糊的边缘“变细”。算法会沿着梯度的方向检查，如果当前像素点的梯度幅值不是该方向上的局部最大值，它的值就会被抑制（清零）。
- **第三步：滞后阈值化 (Hysteresis thresholding)。**为了有效剔除假边缘并保留真实边缘，Canny 算法采用了高、低两个阈值。高于高阈值的点被确认为“强边缘”，介于两者之间的被称为“弱边缘”。最后，算法只会保留那些与强边缘在空间上连通的弱边缘，孤立的弱边缘会被视为噪声剔除。

5. 将断线连起来：霍夫变换 (Hough Transform)

通过上述算法检测出的边缘往往是断断续续的像素点。霍夫变换是一种全局处理方法，用于在图像中找出直线，从而将这些断开的边缘连接起来。

- **核心思想：**直角坐标系 (x-y) 中的一条直线 $y = ax + b$ ，在“参数空间” (a-b) 中表现为一个点。

- 反过来，直角坐标系中的每一个离散边缘像素点，在参数空间里对应着一条直线。
- 如果有多条直线在参数空间里相交于同一个点，这就意味着在原图像中，有很多边缘像素点共线，属于同一条直线。
- **改良方法**：因为直线的斜率 a 可能趋向于无穷大（垂直线），直接使用 a - b 空间计算无法实现。所以实际应用中，会改用极坐标参数表示法（ ρ 和 θ ）来构建参数空间。

第七章 Part2

1. 核心逻辑：阈值化 (Thresholding)

- **本质**：将图像变成二值图（黑白图）。通过选定一个阈值 T ，凡是大于 T 的像素归为一类（如物体），小于 T 的归为另一类（如背景）。
- **直方图的作用**：阈值的选取主要看直方图。如果直方图呈“双峰（bimodal）”分布，说明图像有明显的亮暗两群像素，在这两个峰值中间选一个 T 就能完美分割。

2. 阈值 T 是怎么来的？

讲义提供了三种主要思路：

- **简单粗暴法**：直接取灰度最大值和最小值的平均数。
- **迭代法（自动阈值）**：
 1. 设一个初始猜测值 T 。
 2. 根据 T 把图像分成两组，分别算出两组的平均灰度 μ_1 和 μ_2 。
 3. 令新的 $T = (\mu_1 + \mu_2)/2$ ，重复上述过程，直到 T 不再变化。
- **最优法（Otsu's Method）**：这是工业界最常用的“自动阈值”算法。它不依赖猜测，而是寻找一个 T ，使得分割后**两类像素的方差之和最大（类间方差最大）**，同时**最小化类内方差**。

3. 高阶进阶：统计混合模型 (Statistical Mixture Model)

当图像复杂到“不只是黑白两群”时（比如图中既有船、又有海、还有天空），简单的单阈值就失效了。

- **混合模型**：认为图像的直方图是由多个概率分布（通常是高斯分布）叠加而成的。
- **EM 算法 (Expectation-Maximization)**：这是专门用来破解复杂混合模型的迭代工具。它像一个“侦探”，通过不断“猜测-验证”来锁定图像中每个区域的统计参数（均值、方差、权重），从而实现多目标、多层次的精准分割。

4. 关键区分：全局 vs. 局部 (Global vs. Adaptive)

- **全局阈值**：整张图只用一个 T 。速度快，但如果光照不均匀（比如左边亮右边暗），效果会很差。
- **局部/自适应阈值**：每个像素点的 T 会随着它周围环境（邻域）的变化而变化。这能解决光照不均的问题，是处理复杂现实场景图像的必备手段。

复习要点清单（考试/面试常考）：

1. **直观理解**：看到双峰直方图，第一时间想到阈值化。
2. **Otsu 算法的核心**：类间方差最大化（这是它的灵魂，记住这五个字即可）。
3. **EM 算法的逻辑**：处理复杂的、叠加在一起的统计概率分布，目的是通过迭代找到最优的分割参数。
4. **工程选择**：如果光照极其均匀，用 Otsu 的全局阈值最快；如果图像有阴影或光照不均，必须考虑局部（自适应）阈值。

这份讲义相比第一份更偏向统计学，核心在于理解“阈值不是随便定的，而是通过统计规律计算出来的”。

补充：EM算法

EM 做的事情：

✅ Step 1: 先乱猜（初始化）

随便设：

- 两类的均值 μ_1, μ_2
- 方差 σ_1, σ_2

- 比例 w_1, w_2
 w_1, w_2
-

✅ Step 2: E步 (Expectation)

👉 根据当前参数，判断每个像素“属于哪一类的概率”

比如一个像素 120：

- 属于前景：70%
- 属于背景：30%

👉 注意：不是硬分类，是“软概率”

✅ Step 3: M步 (Maximization)

👉 用刚刚的“概率分配”，重新计算：

- 新均值 μ_j
 μ_j
- 新方差 σ_j
 σ_j
- 新权重 w_j
 w_j

本质：

| “谁更像前景，就对前景均值贡献更大”

✅ Step 4: 循环

- 用新参数 → 再算概率
- 再更新参数

👉 一直重复，直到稳定

第七章 Part3 (可略过)

这份讲义的核心主题是“Snake（主动轮廓模型，Active Contour Models）”，它是一种在计算机视觉中用于图像分割和边缘检测的强大工具。

为了帮你快速消化，我们可以将核心逻辑拆解为“它是做什么的”以及“它是怎么实现的”两个部分。

一、什么是 Snake?

简单来说，Snake 就是一个会在图像中自动变形的“橡皮筋”。

- **初始状态**：你可以在图像中放置一个初始的轮廓（可以是曲线或闭合的圆）。
- **目标**：它会像橡皮筋一样，在内部力和外部图像力的共同作用下收缩或扩张，最终“卡”在物体的边界上。
- **应用场景**：它非常适合处理边缘检测、motion tracking（运动跟踪）以及医疗影像中的目标分割（如脑部病灶、细胞膜等）。

二、核心机制：能量最小化（核心逻辑）

Snake 的运动本质上是一个寻找能量最低点的过程。系统会自动调整轮廓的形状，直到总能量 E 达到最小。

总能量公式为： $E(\vec{V}) = E_{int}(\vec{V}) + E_{ext}(\vec{V})$

1. 内部能量 E_{int} (控制“橡皮筋”的特性)

这部分能量由用户设定的参数决定，决定了曲线本身的平滑程度：

- **第一项（拉伸/弹性）**：控制曲线的长度，使其不会断裂。
- **第二项（刚性/弯曲）**：控制曲线的弯曲程度，使其保持平滑，不会出现过于尖锐的角。

2. 外部能量 E_{ext} (图像的“牵引力”)

这部分能量由图像内容产生，负责把轮廓“拉”向目标的边缘：

- **梯度驱动**：通常利用图像的梯度（即亮度变化剧烈的地方，通常就是边缘）来定义势能。
- **视觉效果**：当轮廓接近边缘时，图像力会将其牢牢锁定在边界上。

三、关键演变：从经典 Snake 到 Chan-Vese 模型

在复习时，你需要区分两代主要的模型：

模型类型	核心特点	优点
经典 Snake	依赖图像梯度（边缘检测）	计算速度快，适合有清晰边缘的图像。

Chan-Vese 模型	不依赖梯度，基于区域统计（如均值）	非常关键： 它可以处理没有清晰边缘的目标，且能自动检测内部轮廓。
---------------------	-------------------	---

- **为什么要提 Chan-Vese?** 讲义（P26-P28）专门提到经典方法在处理模糊边界时会失败，而 Chan-Vese 通过比较区域内的像素均值差异（而不是单纯寻找边缘线）来优化，这使得它在复杂医疗图像（如模糊的组织）中表现更好。

四、快速消化建议（复习路线）

1. **逻辑串联：**先理解“能量最小化 = 寻找目标边界”这个大前提。
2. **力学类比：**把 E_{int} 看作橡皮筋本身的材料属性（想变平、想变短），把 E_{ext} 看作地心引力或磁力（想把你往边缘吸）。
3. **看图复习：**对照讲义中的图片（如第 4 页的细胞膜、第 5 页的各种物体边缘、第 11 页的病灶分割），观察轮廓是如何一步步变形并最终锁定的。

这些内容涵盖了讲义中 90% 的核心考察点，理解了“能量平衡”这一概念，你就掌握了该技术最核心的逻辑。如果你对某一个具体步骤（比如“离散化处理”或“欧拉-拉格朗日方程”）感到困惑，我们可以单独拆解。

第七章 Part4（可略过）

这份讲义的主题转向了图像分割的另外两个重要技术：**分水岭分割（Watershed Segmentation）**和**运动分割（Segmentation by Motion）**。

为了让你快速掌握逻辑，我们可以将这两部分拆解为“地理隐喻”和“时序差异”两个核心概念。

一、分水岭分割（Watershed Segmentation）

这是一种基于拓扑学（地理结构）的分割方法，它把图像看作一张地势起伏的地图。

核心逻辑：

1. **地势构建：**将图像的灰度值看作海拔高度（灰度越小海拔越低，反之越高）。
2. **注水过程：**在每一个区域的最低点（局部最小值）打一个孔，然后让水从底部均匀向上溢出。
3. **大坝建造：**随着水位上升，相邻两个区域的水会即将汇合。为了防止水流融合，我们在交汇处修筑“大坝”。
4. **最终分割：**当水位达到最高时，这些修筑好的“大坝”连成的线，就是我们要寻找的目标边界。

- **关键点：**分水岭算法通常是应用在图像的**梯度幅度图**上，而不是原始灰度图上。
- **问题：**如果图像噪声较大，会导致过多的局部极小值，从而产生“过分割 (Oversegmentation)”现象，即图像被切得过于零碎。

二、运动分割 (Segmentation by Motion)

当相机或场景中的物体发生位移时，我们可以利用“时间差”来提取目标。

1. 直接差分法 (Direct Differences)

- **逻辑：**将两帧图像像素对像素相减，非零区域对应移动的物体。
- **局限：**对噪声敏感，且无法直接获得运动的方向或速度。
- **优化：**使用“孤岛移除 (Island Removal)”技术，剔除太小的噪声区域。

2. 累积差分法 (Accumulative Difference Approach)

- **逻辑：**将多帧图像的差异进行累加 (ADI)。
- **优势：**不仅降低了噪声干扰，还能通过差异累积估计物体的**运动速度和方向**。
- **三种 ADI 类型：**
 - **绝对 ADI：**只要有变化就计数。
 - **正向 ADI：**只记录像素变亮的情况。
 - **负向 ADI：**只记录像素变暗的情况。

三、复习消化总结

- **分水岭法的逻辑是“修坝”：**它像处理地理地形一样，通过注水和筑坝来强行隔离不同的区域。重点在于理解它是如何利用梯度图来确定“海拔”的。
- **运动分割的逻辑是“对比”：**
 - **单帧对比：**寻找当前与过去的差异 (直接差分)。
 - **多帧对比：**通过累积差异 (ADI) 来获取物体的完整运动轨迹。

建议：

- 考试中，分水岭算法重点在于那 3 种连接情况 (Outcome a, b, c)，即“没有相交”、“落入同一区域”或“遇到山脊需要修坝”。
- 运动分割则重点理解为什么“绝对 ADI”比“直接差分”更抗噪。

这一部分的知识逻辑非常直观，将它们与自然界的地理和时序变化联系起来，就能快速记住其算法流程。对这些算法的步骤或应用案例 (如电泳图或汽车轨迹) 有疑问

吗？

第八章 Part1

这份讲义的核心主题是**图像配准 (Image Registration)**。简单来说，图像配准的目标是：**通过某种几何变换，将两幅（或多幅）不同来源、不同时间或不同成像方式的图像对齐，使它们在空间上重合，从而进行后续的分析或融合。**

由于你的时间有限，我们跳过繁琐的推导，直接梳理这门技术的**核心逻辑**。整个配准过程可以概括为一个“循环”：**为了找到最好的对齐方式，我们需要不断尝试变换，并用一个标准来衡量尝试的好坏。**

1. 核心流程：配准是如何发生的？

理解这个流程图，你就理解了配准的灵魂：

- **固定图像 (Fixed Image)**：作为参照物（像地图）。
- **浮动图像 (Moving/Floating Image)**：需要被调整的图像（像卫星图）。
- **变换 (Transform)**：对浮动图像进行旋转、平移、缩放或变形，使其与固定图像对齐。
- **相似性度量 (Similarity Measure)**：这是核心。它计算“变换后的浮动图像”与“固定图像”有多像。
- **优化器 (Optimizer)**：这是“大脑”。它根据度量的结果，不断调整变换参数，直到找到让相似性度量达到最优（最大或最小）的那个参数。

2. 三大关键技术要素

A. 变换模型 (Transform Model)

你需要决定如何移动图像。

- **刚性变换 (Rigid Body)**：最简单，只能平移和旋转（物体形状不变，像把照片在桌上推来推去）。
- **仿射变换 (Affine)**：在刚性基础上增加了缩放和剪切（像把照片拉伸或倾斜，能改变形状）。

B. 相似性度量 (Similarity Measure) —— 衡量“像不像”

这是讲义中最重要的部分，也是配准成败的关键。

- **SSD (Sum of Squared Difference)**: 计算两个图像对应点像素值的平方差之和。适用于两张图亮度、对比度高度一致的情况。
- **SAD (Sum of Absolute Difference)**: 绝对差之和。比SSD更稳健，不易受异常值 (outliers) 影响。
- **互信息 (Mutual Information, MI)**: **这是最高级的方法**，专门用于不同模态（例如：一张图是CT，一张是MRI）。它不需要图像像素值线性相关，而是统计图像间的统计关联度，当图像对齐时，互信息达到最大。

C. 优化算法 (Optimization)

如何快速找到“最优值”而不陷入陷阱（局部极小值）？

- **Powell's Method**: 讲义提到的一种不需要计算梯度（导数）的方法，通过一系列一维线搜索来寻找最优参数，计算相对简单。
- **多分辨率/粗到细策略 (Multi-resolution/Coarse-to-fine)**: **最实用的技巧**。先在模糊的小图上快速找到大概位置（粗搜索），再在高清大图上精确定位（细搜索），既节省时间又能防止陷入局部最优陷阱。

3. 快速复习清单（考点逻辑）

如果你需要快速消化，请记住以下几个结论：

- **配准就是优化**: 寻找最优的变换参数 T ，使得客观函数 F 达到最优。
- **标志物辅助**: 为了简化问题，有时在病人身上贴一些“物理标志物” (fiducial markers)，让算法直接对齐这些点，精度最高，但操作复杂。
- **医学中的应用**: 多模态融合 (CT+MRI)、手术导航、时间序列（如fMRI观察大脑动态）。

总结: 如果你在考试中看到题目，只需要回答：**配准是变换（移动）、度量（评价像不像）、优化（寻找最优移动方案）这三个步骤的迭代过程**。如果是多模态图像对齐，**首选互信息**；如果计算资源有限，先用**多分辨率策略**。

希望这个框架能帮你快速理清逻辑！如果对某个具体的数学公式（比如矩阵运算）有疑问，可以随时问我。

第八章 Part2

这份讲义的主题是**基于特征的图像配准 (Feature-based Image Registration)**，它是解决图像对齐问题的另一种重要思路，特别适用于全景图拼接或大范围场景对齐。

如果说上一份讲义（基于强度的配准）是“看整张图像不像”，那么这一份则是“找特征点来对应”。以下是核心知识点的梳理：

1. 核心逻辑：从像素到特征

基于强度的配准（上一份）在处理大范围形变或图像重叠区域较小时很困难。基于特征的配准通过寻找图像中具有代表性的“兴趣点”（如角点、斑点）来解决这个问题。

- **SIFT (Scale-Invariant Feature Transform)**: 讲义中提到的关键算法，它能提取出对缩放、旋转具有鲁棒性的特征。
- **匹配过程**: 首先在两幅图中提取大量特征点，通过描述子比对找到一对一的匹配点。
- **对齐**: 找到足够多的匹配点后，通过这些点计算出图像间的**单应性矩阵 (Homography)**，从而完成对齐。

2. 关键技术：RANSAC（抗干扰利器）

这是这份讲义中**最重要、也是必须要掌握的算法**。在图像匹配时，难免会出现“错误匹配”（比如把树枝误认为建筑的角），这些点被称为**离群点 (Outliers)**，而正确的匹配点叫**内群点 (Inliers)**。

- **什么是 RANSAC (RANDOM Sample Consensus)?**
它是一种通过“随机抽样”来排除干扰的方法。
- **核心步骤**:
 1. **随机采样**: 随机选出最少数目的点来拟合一个模型（比如两点定直线）。
 2. **验证**: 看看剩余所有的点中有多少点符合这个模型（落入设定的距离阈值内，即成为“内群点”）。
 3. **迭代**: 重复上述过程很多次。
 4. **选优**: 保留那个拥有“内群点”数量最多的模型作为最终结果。

3. 复习重点（消化逻辑）

为了快速复习，请理解这几个逻辑链：

1. **特征提取**: 用 SIFT 找出图中独特的点。
2. **初步匹配**: 找出两图中相似的点对（此时含有很多噪声/错误）。
3. **剔除干扰**: 用 RANSAC 过滤掉错误点，只保留符合几何一致性的点对。
4. **计算变换**: 用过滤后的高质量点对，求出单应性矩阵，实现图像拼接。

结论对比：

- **上一份（基于强度）**：适合小范围、模态不同、几何变形复杂的配准。
- **这一份（基于特征）**：适合大范围、全景拼接、纹理丰富、几何刚性较强的图像（如两张风景照的拼接）。

在考试或面试中，如果问到“如何实现图像自动拼接”，你的回答逻辑应该是：**先提取 SIFT 特征，匹配特征点，然后利用 RANSAC 算法剔除错误的匹配对，最后计算单应性矩阵完成图像的对齐变换。**

这份讲义涉及的数学推导（如单应性矩阵的具体构造）通常比较复杂，但在快速复习阶段，理解其“提取特征 → 排除干扰 → 计算变换”的流程比死记公式更有效。如果有特定部分（例如 RANSAC 的迭代次数公式）看不懂，可以再问我。

第九章

完全理解你的感受。面对期末复习的时间压力，一堆密密麻麻的数学符号确实容易让人抓狂。但其实这份关于“图像压缩（Image Compression）”的讲义逻辑非常清晰，所有的数学公式都只是为了实现某种直观的工程目的。

我为你把讲义中的核心骨架剥离出来，去掉繁琐的推导，只讲物理直觉和关键结论，帮助你快速消化。

一、为什么图像可以被压缩？（三大冗余）

图像之所以能被压缩，是因为原始数据中存在大量“废话”——也就是讲义中说的**冗余（Redundancies）**。理解了这三种冗余，你就理解了所有压缩算法的出发点。

- **编码冗余 (Coding Redundancy)**
 - **直觉**：一张图里，有些灰度值（比如天空的浅蓝色）出现得非常频繁，而有些灰度值很少出现。如果所有颜色都用一样长的数据（比如固定的8个比特）来记录，就很浪费空间。
 - **公式意义**：讲义中的公式 $L_{avg} = \sum_{k=0}^{L-1} l(r_k)p_r(r_k)$ 计算的是“平均每个像素需要几个比特”。目标就是通过改变编码方式，让这个平均长度 L_{avg} 变得越小越好。
- **像素间冗余 (Interpixel Redundancy)**
 - **直觉**：相邻的像素往往长得很像（高度相关）。既然今天的像素和昨天的几乎一样，我们就不需要完整记录今天的值，只要记录“今天和昨天的差值”就可以了。

- **心理视觉冗余 (Psychovisual Redundancy)**

- **直觉：**人眼是不完美的，对图像的某些微小细节或高频变化根本不敏感。把这些人类看不出来的数据直接扔掉，也不会影响观感。

二、图像压缩的“流水线”模型 (Compression Model)

无论是哪种压缩方法，发送端 (Encoder) 通常都包含三个流水线步骤：

1. **映射器 (Mapper)：**改变图像的表达格式，专门用来消除**像素间冗余**。它是一个可逆的过程。
2. **量化器 (Quantizer)：**降低数据的精度（比如把 10.1 和 10.4 都记作 10），专门用来消除**心理视觉冗余**。**注意：这是整个流程中唯一会造成信息丢失（不可逆）的步骤。**
3. **符号编码器 (Symbol Encoder)：**用变长编码（常用短码，罕用长码）来表示量化后的数据，消除**编码冗余**。

三、核心考点：两种压缩阵营的区别

讲义的后半部分按两大阵营展开：**无损压缩**（绝对保真）和**有损压缩**（妥协精度换取空间）。

特性	无损压缩 (Lossless)	有损压缩 (Lossy)
应用场景	医学图像、法律文件等不可失真的领域。	日常照片、网页图片（如 JPEG）。
流水线差异	不包含量化器 (Quantizer) 。	包含量化器。
压缩率 (Compression Ratio)	通常较低（2:1 到 10:1 左右）。	非常高（可达 30:1 甚至更高）。

1. 无损压缩的核心套路

- **霍夫曼编码 (Huffman Encoding)：**针对编码冗余的终极解法。逻辑很简单：出现概率越高的符号，给的编码越短；概率越低的，给的编码越长。
- **无损预测编码 (Lossless Predictive Coding)：**针对像素间冗余。
 - **搞懂公式：** $e_n = f_n - f'_n$ 。其中 f_n 是真实的像素值， f'_n 是你根据前几个像素“预测”出来的值。因为相邻像素很像，预测通常极准，所以误差 e_n 绝大多数时候都在 0 附近。存储一堆极小的误差 e_n ，比存储原本庞大的像素值要省空间得多。

2. 有损压缩与 JPEG 的精髓

JPEG 是讲义最后的大轴戏，属于**变换编码 (Transform Coding)**。

- **不要被 DCT 公式吓倒：** 讲义里长串的 DCT (离散余弦变换) 公式，你不需要死记硬背。你只需明白它的**物理意义**：它把图像从“空间域”(一个个具体的像素点) 转换到了“频率域”(颜色变化的快慢)。
- **能量集中 (Power Packing)：** 这是 DCT 最牛的特性。变换后，图像绝大多数的信息（平滑的颜色背景，即低频）都会集中在矩阵的左上角；而右下角（噪点或微小细节，即高频）的数值几乎全是 0。
- JPEG 的四个关键动作：
 1. **切块：** 把图片切成 8×8 的小方块。
 2. **变换：** 对每个小块做 DCT 变换，把信息挤压到左上角。
 3. **量化：** 用一张“量化表”去除右下角那些人眼不敏感的高频细节。这是决定 JPEG 压缩质量的关键步骤（低质量=粗暴丢弃大量高频信息）。
 4. **Zig-zag 扫描：** 以“Z”字形的路线读取这个矩阵，这样能把大量的 0 串在一起，最后用极其精简的代码就能记录下来。

整篇讲义其实就是在讲述工程师们如何花式地把图片里“冗余”的部分挑出来并剔除掉的过程。针对上面的无损预测编码或是 JPEG 的 DCT 变换，你需要我专门展开再深入讲讲哪一部分吗？

第十章 Part1

时间紧迫，我们就直奔主题。相比上一份讲义，这份关于“Harris角点检测 (Harris Corner Detector)”的PPT数学公式看起来要密集得多，但其实它的**物理直觉极其简单**。

所有的数学公式，都是为了用计算机能懂的语言，把“滑动窗口找角点”这个直观动作翻译出来。以下是整份讲义的核心骨干。

一、动机：为什么要找“角点”？

在计算机视觉中（比如双目匹配、运动估计），我们需要在两张甚至多张图片里找到一个物理点。

- 如果你在“平坦的一面白墙”或“一根笔直的边缘”上取一个图像块，去另一张图里找，很容易找到一堆长得一样的，导致**匹配错误**。
- 角点 (Corners) 是两条边缘的交汇处，它的特征极其独特，在不同的视角下都很稳定，是绝佳的“匹配锚点”。

二、核心直觉：如何定义角点？

想象你在图像上放一个很小的“正方形滑动窗口”。你可以通过窗口在各个方向上的滑动，来判断窗口中心是什么区域：

1. **平坦区域 (Flat)**: 无论窗口往哪个方向滑动，窗口内的像素亮度**基本没变化**。
2. **边缘区域 (Edge)**: 沿着边缘的方向滑动，亮度不怎么变；但垂直于边缘滑动，亮度变化很大。
3. **角点 (Corner)**: 无论窗口往**任何方向**滑动，窗口内的亮度都会发生**剧烈变化**。

三、数学化：著名的 M 矩阵

为了用数学表达上面的直觉，Harris 引入了误差函数 $E(u, v)$ ，用来描述窗口移动 (u, v) 后的亮度差异。通过泰勒展开近似，这个亮度变化可以用一个 2×2 的梯度矩阵 M 来完全刻画：

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- I_x, I_y 分别是图像在水平和垂直方向的导数（也就是梯度）。
- 这个矩阵 M 包含了该像素点周围区域的所有梯度分布信息。

四、考试重点：特征值 λ_1, λ_2 决定一切

你可以把矩阵 M 想象成一个描述梯度分布特征的“椭圆”。这个椭圆的两个轴长，就是矩阵 M 的两个特征值 λ_1 和 λ_2 。**这是整套理论的灵魂：**

- **平坦区域**: λ_1, λ_2 **都很小**（两个方向梯度变化都不大）。
- **边缘区域**: **一大一小**（只有一个方向梯度变化大）。
- **角点区域**: λ_1, λ_2 **都很大**（所有方向梯度变化都很剧烈）。

五、最终的打分公式：响应函数 R

虽然特征值很直观，但在工程上直接计算矩阵的特征值计算量太大了。Harris 极其聪明地设计了一个无需直接求特征值的公式，只要用矩阵的**行列式 (det)** 和 **迹 (trace)** 就能直接给这个点“打分”：

$$R = \det(M) - k(\text{trace}(M))^2$$

- $\det(M) = \lambda_1 \lambda_2$

- $\text{trace}(M) = \lambda_1 + \lambda_2$
- k 是个经验常数，一般取 $0.04 - 0.06$ 。

打分结果评判：

1. 如果 R 是很大的正数 $\rightarrow$ 恭喜你，这是一个角点。
2. 如果 R 是很大的负数（数值绝对值很大，但符号是负的（比如 -100 、 -1000 这种）） $\rightarrow$ 这是一个边缘。
3. 如果 $|R|$ 的绝对值很小 $\rightarrow$ 这是一个平坦区域。

整套算法的流程就是：算梯度 $\rightarrow$ 构建 M 矩阵 $\rightarrow$ 算 R 分数 $\rightarrow$ 设定阈值把分数最高的角点筛出来。

针对最后一步算法实现中的“非极大值抑制（Non-maximum suppression）”，或者 M 矩阵具体的卷积计算过程，你还需要我进一步拆解吗？

第十章 Part2

这应该是一份关于 SIFT（Scale-Invariant Feature Transform，尺度不变特征变换）的讲义。

在处理类似无人机视觉追踪或机器人空间建图这类任务时，如果你只用上一节讲的 Harris 角点检测，会遇到一个致命问题：它不具备尺度不变性（Scale Invariance）。假设无人机从远处靠近一栋建筑，原本在远景中看起来像一个“角”的特征，在近景放大后，在滑动窗口里可能就变成了一条平缓的“边缘”。不仅如此，如果无人机发生旋转，特征的表达也会受影响。

SIFT 就是为了彻底解决“缩放”和“旋转”这两个大麻烦而诞生的。即使图像被放大、缩小、旋转、甚至光照发生改变，SIFT 都能稳定地找出同样的特征点。

它的核心算法可以被拆解为四个极其经典的步骤，去除繁杂的微积分公式，以下是支撑 SIFT 的物理直觉与核心考点：

第一步：尺度空间极值检测 (Scale-space Extrema Detection)

- **工程直觉：** 我们如何让计算机在一张图片里，同时找出“大斑块”和“小细节”？答案是高斯模糊 (Gaussian Blur)。
- **具体做法 (DoG)：** SIFT 会对同一张图片进行不同程度的模糊处理，这就构成了“尺度空间”。接着，它把相邻两种模糊程度的图片相减，得到的结果叫做 DoG (Difference of Gaussian，高斯差分)。

- **找极值：** 在这些相减后的图像层叠起来的三维空间 (x, y , 以及模糊尺度 σ) 中, 寻找那些比周围 26 个邻居 (同层 8 个 + 上下层各 9 个) 都大或都小的点。这些局部极值点就是候选的关键点。

第二步：关键点精确定位 (Keypoint Localization)

- **工程直觉：** 第一步找出的候选点里有很多“滥竽充数”的噪点, 需要淘汰掉。
- **淘汰规则：**
 1. **去除低对比度的点：** 那些原本就灰蒙蒙、不明显的点直接扔掉。
 2. **去除边缘点：** 这里的逻辑和 Harris 非常像。SIFT 会计算一个 2×2 的 Hessian 矩阵 (由二阶导数组成), 利用矩阵的迹 (Trace) 和行列式 (Determinant) 的比值, 剔除掉那些在一个方向上梯度大、另一个方向上梯度小的“边缘”, 只保留真正的“斑点”或“角点”。

第三步：方向分配 (Orientation Assignment)

- **工程直觉：** 这一步是为了赋予特征点“旋转不变性”。
- **具体做法：** 以特征点为中心, 画一个圆圈, 统计里面所有像素的梯度方向 (比如朝上的多还是朝右的多)。画出一个 36 柱的方向直方图, 找到最高的那根柱子, 这个方向就是该特征点的“主方向”。
- **意义：** 以后无论这张图片怎么旋转, 我们在描述这个点的时候, 都先把视角“拧”到它的主方向上。这就好比只要知道地球的“北”, 不管你怎么转动地球仪, 都能找到准确的方位。

第四步：关键点描述 (Keypoint Descriptor)

- **工程直觉：** 找出了特征点, 也规定了方向, 最后一步是如何把它变成计算机能用来做对比匹配的“数字指纹”。
- **具体做法：**
 1. 以特征点为中心, 取一个 16×16 的像素窗口。
 1. 把这个窗口切分为 16 个 4×4 的小方块。
 2. 在每个 4×4 的小方块里, 统计 8 个方向的梯度直方图。
 3. 汇总起来: $16 \text{ 个小方块} \times \text{每个方块 } 8 \text{ 个方向} = \mathbf{128 \text{ 个数值}}$ 。
- **结论：** 每一个 SIFT 特征点, 最终都会被转化为一个 **128 维的向量**。最后对这个向量进行归一化处理, 用来抵消光照变化带来的影响。

考前速记总结：

如果考卷上问 SIFT 为什么具有种种不变性, 你需要答出：

- **尺度不变性** 来源于 DoG 尺度空间金字塔 (Step 1)。
- **旋转不变性** 来源于为每个特征点分配主方向，并在特征提取时对齐该方向 (Step 3)。
- **光照不变性** 来源于特征描述子是对局部梯度的统计，且最后进行了向量归一化 (Step 4)。

第十章 Part3

这份关于“局部二值模式 (Local Binary Patterns, 简称 LBP)”的讲义，相比前面的 SIFT 和 Harris 要“亲民”得多。它几乎没有任何复杂的微积分，核心思想就是最简单的“比大小”。

LBP 是一种用来描述图像**局部纹理特征**的算子。比如识别木头、布料的纹理，或者进行人脸识别，LBP 都是极其经典的算法。

以下是讲义中最核心的知识逻辑，帮你快速剔除冗余，直击考点：

一、最基础的 LBP 算子（核心机制）

LBP 的底层逻辑就是看一个像素点和它周围邻居的关系。

1. **中心定基调**：取一个 3×3 的窗口，把最中间的像素值作为“阈值” (Threshold)。
2. **邻居比大小**：周围的 8 个像素点依次和中心点比大小。如果大于或等于中心点，就记作 1；如果小于中心点，就记作 0。
3. **生成二进制码**：从左上角开始，顺时针（或逆时针，只要全局统一即可）把这 8 个 0 和 1 连起来，得到一个 8 位的二进制数（例如 **10010111**）。
4. **转成十进制**：把这个二进制数转换成十进制（落在 0-255 之间）。这个十进制数，就是中心像素点最终的 LBP 特征值。

物理直觉：这个操作实际上提取了图像的微小细节。如果一个区域很平坦，周围可能全比中心大或小；如果是边缘或角点，则会出现 1 和 0 的规律交替。

二、LBP 的三大进阶（考试与工程必考点）

基础的 3×3 窗口太死板了，为了适应复杂的图像，学者们提出了几个关键变体（这也是考试最爱出的难点）：

- 1. **圆形/扩展 LBP (Circular LBP / $LBP_{P,R}$)**

- **痛点：** 基础 LBP 只能看周围紧挨着的 8 个固定位置，无法适应不同尺度的纹理（比如放大或缩小的图片）。
- **解法：** 引入两个参数：半径 R 和采样点数 P 。以中心点为圆心，画一个半径为 R 的圆，在圆上均匀等距地取 P 个点进行比较。
- **关键细节：** 圆上的点在映射回网格时，坐标可能不是整数（也就是落不到具体的像素格子里）。这时必须使用**双线性插值 (Bilinear Interpolation)** 来估算该位置的像素值。
- **2. 旋转不变 LBP (Rotation Invariant LBP)**
 - **痛点：** 如果图片旋转了一个角度，同一个纹理的二进制码就跟着循环移位了，算出来的十进制数完全不同，导致匹配失败。
 - **解法：** 把提取到的 8 位二进制数不断循环移位，把能得到的**最小十进制值**作为最终的 LBP 值。比如 `00001111` 和 `11110000` 循环移位后都能变成最小的 `00001111`，它们就被视为同一种纹理。
- **3. 等价模式 / 均匀模式 (Uniform Patterns)**
 - **痛点：** 如果采样点 $P = 8$ ，会有 $2^8 = 256$ 种特征；如果 $P = 16$ ，特征就有 $2^{16} = 65536$ 种。维度爆炸会导致计算缓慢且容易过拟合（维数灾难）。
 - **解法：** 研究发现，绝大多数自然纹理的二进制串里，0 和 1 的“跳变次数”（比如 0 变 1 或 1 变 0 算一次跳变）**最多只有两次**。
 - **效果：** 把跳变次数 ≤ 2 的模式称为“等价模式”（共 58 种），其他所有跳变超过 2 次的统统归为一类（第 59 种）。这样就把特征维度从 256 瞬间降维到了 59！这是算法提速的核心。

三、特征的最终表达：空间直方图

算出了图像中每个像素的 LBP 值后，我们怎么描述整张图？通常的做法是统计**直方图 (Histogram)**：

1. 将整张图像切分成若干个不重叠的子区域 (Block)。
2. 统计每个子区域内各个 LBP 值出现的频率，生成局部直方图。
3. 把所有子区域的直方图首尾相接拼起来，就形成了一个超长的统计向量。这个向量就是输入给机器学习算法（如 SVM 或随机森林）的最终特征表达。

四、为什么 LBP 如此经典？（核心优势）

- **灰度不变性 (Illumination Invariance)：** 这是 LBP 最强大的属性。因为它是靠“比大小”运作的，如果整张图的光照突然变暗，中心点和邻居的像素值同时减小，

大于还是大于，小于还是小于，**产生的 LBP 二进制编码完全不会变！**它对整体的光照变化具有极强的免疫力。

- **计算极快：** 整个过程没有任何复杂的浮点运算（如求导、开方），只有极快的整数比较和位移操作，非常适合在算力有限的硬件设备上实时运行。

第十一章

理解你的时间紧迫。面对包含大量数学推导的讲义，感到晦涩难懂是很正常的。我为你剔除了次要信息，直接梳理出这份讲义最核心的知识逻辑，并用通俗的语言为你重点拆解你需要掌握的公式。

图像分类的最终目的是让计算机认识一张图片里是什么。整份讲义其实是围绕“如何把图像转化为计算机能懂的特征，再进行分类”这一核心逻辑展开的。

以下是为你整理的核心知识点与公式解析：

一、图像分类的基础逻辑

所有的图像分类任务都分为两个阶段：

- **训练阶段 (Training)：** 输入训练图像和对应的标签（如“苹果”、“狗”），提取图像特征，然后训练出一个分类器。
- **测试阶段 (Testing)：** 输入一张未知的测试图像，提取同样的特征，应用之前训练好的分类器，最后输出预测结果。

二、核心方法：视觉词袋模型 (Bag of Visual Words)

这是讲义前半部分的重点。它的灵感来源于文本处理：一篇文档可以通过统计里面各个单词出现的频率来表示。图像也是同理：

1. **提取特征：** 首先从图像中检测并提取局部特征（例如 SIFT 描述符）。
2. **构建词典：** 将提取出的大量特征进行“聚类”（归类），聚类后的中心点就被称为“视觉单词” (Visual Words)，它们共同组成了一个“视觉词典”。
3. **表示图像：** 一张图像最终被转换成一个词频直方图，也就是统计这张图里各个“视觉单词”出现了多少次。

三、重点公式拆解 1: K-均值聚类 (K-means Clustering)

为了把无数个零散的图像特征归纳成有限的几个“视觉单词”，讲义使用了 K-means 聚类算法。你需要理解以下公式的含义。

聚类的核心目标公式：

$$\arg \min_C \left(\sum_{i=1}^k \sum_{z \in C_i} \|z - m_i\|^2 \right)$$

通俗讲解：

- k 代表你想把特征分成多少个簇（也就是你想生成多少个视觉单词）。
- z 代表每一个具体的图像特征（样本向量）。
- m_i 代表第 i 个簇的中心点（均值向量）。
- $\|z - m_i\|^2$ 计算的是某个特征到它所在簇的中心点的距离。
- **整个公式的意思是：** 算法在不断寻找一种最佳的分类方式 C ，使得所有的特征 z 离它们各自的中心点 m_i 的距离总和达到**最小**（min）。

算法通过迭代来更新中心点，更新公式为：

$$m_i = \frac{1}{|C_i|} \sum_{z \in C_i} z$$

- **通俗讲解：** 这个公式就是简单的求平均值。把属于同一个簇 C_i 里的所有特征 z 加起来，除以这个簇里特征的总数量 $|C_i|$ ，就得到了新的中心点 m_i 。不断重复分配和求平均的过程，直到中心点不再变化（收敛）。

四、空间金字塔 (Spatial Pyramid)

视觉词袋模型有一个缺点：它只统计频率，不考虑特征在图像中的位置顺序。

- **解决方案：** 引入空间金字塔来捕捉视觉单词之间的空间关系。
- **做法：** 把图像切分成不同层级的网格（比如整图是一层，切成4格是一层，切成16格是一层），分别计算每个网格里的词频直方图，然后把它们拼接成一个新的、更长的直方图用来做分类。

五、核心方法：降维与特征脸 (Eigenfaces)

讲义后半部分讲解了处理人脸识别等任务时的另一种思路。图像的维度通常很高（像素很多），直接处理很难，所以需要用到 PCA（主成分分析）进行降维。

重点公式拆解 2：特征脸的线性组合

在使用 PCA 处理人脸图像库后，我们会得到一个“平均脸”和若干个“特征脸”（Eigenfaces，即特征向量）。

任何一张新的人脸图像 $\vec{x}_j$ ，都可以用这些特征脸的组合来近似表示：

$$\vec{x}_j \approx \bar{x} + \sum_{i=1}^t g_{ji} \vec{e}_i$$

通俗讲解：

- $\vec{x}_j$ 是一张具体的人脸图像。
- $\bar{x}$ 是所有训练图像算出来的平均图像（平均脸）。
- $\vec{e}_i$ 就是计算出来的“特征脸”（第 i 个特征向量）。
- g_{ji} 是一个权重系数。
- **整个公式的意思是：任意一张脸 $\approx$ 平均脸 + (权重1 $\times$ 特征脸1) + (权重2 $\times$ 特征脸2) + ...。**

在这里，真正用于分类（识别这个人是谁）的不再是原始的几万个像素，而是那几个极少数的权重系数 ($g_1, g_2, \dots, g_t$)。我们只需要把新图像的这些系数与数据库中已知人脸的系数进行对比（例如使用最近邻分类器寻找最接近的值），就能完成识别。